SOFTWARE REQUIREMENTS SPECIFICATION

FlowSentinel

Explainable network intrusion detection from flow metadata

Version 1.0 | 27 October 2026 | Status: Baseline for development

Document	Software Requirements Specification (SRS)
Product	FlowSentinel, an open-source explainable network intrusion detection system
Version / date	1.0 | 27 October 2026
Author and owner	Prabhat (GitHub: 13-Prabhat)
Repository	github.com/13-Prabhat/flowsentinel (MIT licence)
Standard followed	ISO/IEC/IEEE 29148:2018 structure (supersedes IEEE 830-1998)
Document status	Baseline. Requirement status values reflect the repository on the document date.

Revision history

Version	Date	Author	Description
0.1	14 October 2026	Prabhat	Initial project idea and four-month plan
0.5	21 October 2026	Prabhat	Parser implemented; engineering requirements added
1.0	27 October 2026	Prabhat	First complete baseline: functional, non-functional, interface, data and verification requirements

How to read this document

- Section 1 and 2 explain what the product is and who it is for.
- Sections 5 and 8 hold the numbered, testable requirements. Each row has an ID, priority, verification method, target release and current status.
- Appendix A contains worked numerical examples (header decoding, entropy, metrics, capacity planning) that give the requirement thresholds their reasoning.

Contents

1. Introduction 5

1.1 Purpose 5

1.2 Document conventions 5

1.3 Intended audience 5

1.4 Product scope 5

1.5 Definitions, acronyms and abbreviations 7

1.6 References 7

1.7 Document overview 8

2. Overall description 8

2.1 Product perspective 8

2.2 Product functions (summary) 8

2.3 User classes and characteristics 9

2.4 Operating environment 9

2.5 Design and implementation constraints 9

2.6 Assumptions and dependencies 9

2.7 Release plan 10

2.8 Risk register 10

3. Use cases 11

4. System architecture 12

4.1 Logical architecture 12

4.2 Components 12

4.3 Repository layout (target) 13

4.4 Data flow 13

4.5 Live-mode concurrency 13

5. Functional requirements 14

5.1 Packet acquisition (FR-CAP) 14

5.2 Packet parsing (FR-PAR) 14

5.3 Flow assembly (FR-FLW) 15

5.4 Feature extraction (FR-FEA) 15

5.5 Rule-based detection (FR-DET) 16

5.6 Machine-learning detection (FR-ML) 16

5.7 Explainability (FR-EXP) 17

5.8 Alert management and storage (FR-ALR) 17

5.9 Command-line interface (FR-CLI) 17

5.10 REST API (FR-API) 18

5.11 Dashboard (FR-UI) . . . 18
5.12 Configuration (FR-CFG) . . . 18
5.13 Attack lab (FR-LAB) . . . 19
5.14 Evaluation and benchmarking (FR-EVL) . . . 19
5.15 Monitoring and logging (FR-MON) . . . 19
5.16 Engineering process requirements (FR-DEV) . . . 19
5.17 Packaging and distribution (FR-PKG) . . . 20
6. External interface requirements . . . 21
6.1 User interfaces . . . 21
6.2 Hardware interfaces . . . 21
6.3 Software interfaces . . . 21
6.4 Communication interfaces . . . 22
6.5 File formats . . . 22
7. Data requirements . . . 23
7.1 Packet (implemented) . . . 23
7.2 Flow record . . . 23
7.3 Feature schema version 1 . . . 23
7.4 Alert record and database . . . 24
7.5 Model metadata . . . 24
7.6 Retention, privacy and integrity . . . 24
8. Non-functional requirements . . . 25
8.1 Performance efficiency (NFR-PERF) . . . 25
8.2 Reliability (NFR-REL) . . . 25
8.3 Detection quality (NFR-ACC) . . . 25
8.4 Security (NFR-SEC) . . . 26
8.5 Ethics and legal (NFR-ETH) . . . 26
8.6 Usability (NFR-USE) . . . 26
8.7 Maintainability (NFR-MNT) . . . 27
8.8 Portability and interoperability (NFR-PRT) . . . 27
8.9 Scalability and resource bounds (NFR-SCL) . . . 27
8.10 Quality attribute priorities . . . 27
9. Verification and acceptance . . . 28
9.1 Test strategy . . . 28
9.2 Test data . . . 28
9.3 Acceptance tests . . . 28
9.4 Definition of done (every release) . . . 29
10. Traceability . . . 29

10.1 Objectives to requirement groups . . . 29
10.2 Requirement groups to implementation and verification artefacts . . . 29
Appendix A. Worked numerical examples . . . 30
A.1 Decoding a TCP SYN by hand . . . 30
A.2 Port entropy separates normal hosts from scanners . . . 30
A.3 Why accuracy is not enough . . . 30
A.4 Isolation Forest anomaly score . . . 31
A.5 Capacity and memory planning . . . 31
A.6 False-alarm volume and analyst workload . . . 31
A.7 Drift detection with the Population Stability Index . . . 31
A.8 Fixture file size . . . 31
A.9 Flow key and timeouts . . . 32
Appendix B. Sample configuration (defaults) . . . 32
Appendix C. Sample alert and API payloads . . . 33
Appendix D. Open issues and decisions to confirm . . . 34
Appendix E. Requirements summary . . . 35

1. Introduction

1.1 Purpose

This document specifies the software requirements for **FlowSentinel 1.0**, an open-source network intrusion detection system (NIDS) that finds attacks from the *behaviour* of network flows (packet sizes, timing, direction, flags, entropy) rather than from packet payloads. Detection uses transparent rules and machine learning, and every alert carries a plain-language explanation.

The document is the single reference for what the system must do, which qualities it must have, and how each requirement will be verified. Developers use it to build, testers to derive test cases, and reviewers (including recruiters and open-source contributors) to judge scope and quality.

1.2 Document conventions

Requirement keywords: **shall** = mandatory, **should** = recommended, **may / could** = optional. Functional requirement IDs have the form `FR-GROUP-NN` (the group code is shown in each section title); non-functional IDs have the form `NFR-GROUP-NN`. Numbers inside a group are assigned in table order.

Column	Values and meaning
Pri (priority)	**M** Must: the release is blocked without it. **S** Should: important, may slip with a written reason. **C** Could: desirable if time allows.
Ver (verification)	**T** Test: automated test. **I** Inspection: review of code, configuration or documentation. **A** Analysis: calculation or measured report. **D** Demonstration: run it and observe.
Rel (release)	**R1** v0.1.0, month 1 (mid-Oct to mid-Nov 2026). **R2** v0.2.0, month 2. **R3** v0.3.0, month 3. **R4** v1.0.0, month 4 (to mid-Feb 2027). **Later** after 1.0.
Status	**Implemented**: code and tests exist. **In progress**: being built or awaiting merge. **Planned**: not started.

1.3 Intended audience

Reader	Use of this document	Start with
Developer / contributor	Understand scope, interfaces and the definition of done	Sections 2, 4, 5, 7
Tester	Derive test cases and acceptance tests	Sections 5, 8, 9, 10
Security analyst (user)	Know what the tool detects, its limits and how alerts are explained	Sections 1.4, 3, 5.5 to 5.8
Reviewer / recruiter	Judge engineering rigour and project scope	Sections 1, 2.7, 9, Appendix E

1.4 Product scope

Product name: FlowSentinel. **Problem:** most network traffic is now encrypted, so payload inspection (classic signature matching) sees little. Flow metadata remains visible and is enough to expose scans, floods and brute-force attempts. Analysts also suffer from alert fatigue: a detector that cannot say why it fired, or that fires too often, is not used.

Measurable objectives

ID	Objective	Measure
O1	Detect port scans, SYN floods and SSH/FTP brute force from flow metadata	Recall at least 0.95 on lab attacks; at most 0.1% of benign host windows alert (NFR-ACC-01, -02)
O2	Explain every alert in plain language	100% of alerts carry at least one reason with value and threshold (FR-EXP-01, -02)
O3	Make every result reproducible	Docker lab, fixed seeds, versioned feature schema, reports with library versions
O4	Process traffic fast enough to be useful	At least 20,000 packets/s in pure Python; parse stage 5 times faster with C++ (NFR-PERF-01, -02)
O5	Demonstrate professional engineering	CI on 3 Python versions, pre-commit, 80% coverage, ADRs, Conventional Commits

In scope

- Offline analysis of classic pcap captures and live capture on one network interface (single node).
- IPv4 over Ethernet with TCP, UDP and ICMP; flow assembly; flow and host-window features.
- Rule-based and machine-learning detection with explanations; alert storage, export and query.
- Command-line interface, REST API and a local web dashboard; Docker attack lab; evaluation and benchmarking tools.

Out of scope

- Deep packet inspection, signature matching and TLS decryption.
- Intrusion *prevention* (blocking traffic) and any active response.
- Distributed multi-sensor deployment, high-availability clustering, 10 Gbit/s line rate.
- IPv6 and pcapng in version 1.0 (listed as Later); malware analysis; user-account management beyond an API token.

1.5 Definitions, acronyms and abbreviations

Term	Meaning
5-tuple	Protocol, source address, source port, destination address and destination port; identifies a conversation.
ACK / SYN / FIN / RST / PSH / URG	TCP control flags: acknowledge, synchronise (open), finish (close), reset, push, urgent.
BPF	Berkeley Packet Filter; the syntax of capture filters such as 'tcp port 80'.
Brute force	Repeated login attempts against a service such as SSH or FTP.
CI	Continuous integration: automated checks on every push and pull request.
Conventional Commits	A commit message format: type(scope): summary.
ECS	Elastic Common Schema, a field naming convention for security logs.
Entropy (Shannon)	H = -sum(p log2 p) in bits; measures how spread out values (such as destination ports) are.
Flow	All packets of one 5-tuple conversation in both directions, between a start and an end.
FPR / TPR	False positive rate = FP/(FP+TN); true positive rate (recall) = TP/(TP+FN).
IAT	Inter-arrival time: time between consecutive packets.
Isolation Forest	Unsupervised anomaly detector that isolates unusual points with few random splits.
NIDS / IPS	Network intrusion detection system / intrusion prevention system.
Npcap / libpcap	Packet capture libraries for Windows / Linux and macOS.
pcap / pcapng	Capture file formats; classic pcap is the older, simpler format.
PSI	Population Stability Index; measures how far a feature distribution has drifted.
ROC-AUC / PR-AUC	Area under the receiver operating characteristic / precision-recall curve.
SHAP	SHapley Additive exPlanations; per-feature contribution to a model prediction.
SOC	Security operations centre; the team that triages alerts.
SPAN / mirror port	Switch feature that copies traffic to a monitoring port.
SRS	Software Requirements Specification (this document).
TSO	TCP segmentation offload; the network card splits large segments, so captures on the sender show IP length 0.
TTL	Time to live; hop limit in the IPv4 header.

1.6 References

- ISO/IEC/IEEE 29148:2018, Systems and software engineering: Life cycle processes: Requirements engineering.
- RFC 791 (Internet Protocol), RFC 768 (UDP), RFC 792 (ICMP), RFC 9293 (Transmission Control Protocol).
- libpcap savefile format, tcpdump.org (classic pcap file layout).
- Sharafaldin, Lashkari and Ghorbani. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization (CIC-IDS2017). ICISSP 2018.
- Moustafa and Slay. UNSW-NB15: a comprehensive data set for network intrusion detection systems. MilCIS 2015.
- Liu, Ting and Zhou. Isolation Forest. IEEE ICDM 2008.
- Lundberg and Lee. A Unified Approach to Interpreting Model Predictions (SHAP). NeurIPS 2017.
- Conventional Commits 1.0.0; Semantic Versioning 2.0.0.
- FlowSentinel repository documents: docs/parser.md (parser design and limits).

1.7 Document overview

Section 2 describes the product in context. Section 3 gives use cases. Section 4 gives the logical architecture. Section 5 lists functional requirements by module. Section 6 specifies external interfaces, Section 7 the data, Section 8 the quality attributes. Section 9 explains verification and acceptance, Section 10 traces requirements to artefacts. The appendices hold worked examples, sample configuration and payloads, open issues and summary statistics.

2. Overall description

2.1 Product perspective

FlowSentinel is a self-contained, open-source Python package with a command-line interface and optional services (REST API and dashboard). It does not replace mature engines such as Suricata or Zeek. It complements them by offering **explainable, flow-based machine-learning detection** in a small code base that is easy to read, test and extend.

It consumes capture files written by Wireshark or tcpdump, or sniffs one interface (optionally a SPAN port), and produces alerts in SQLite, JSON Lines, the REST API and the dashboard.

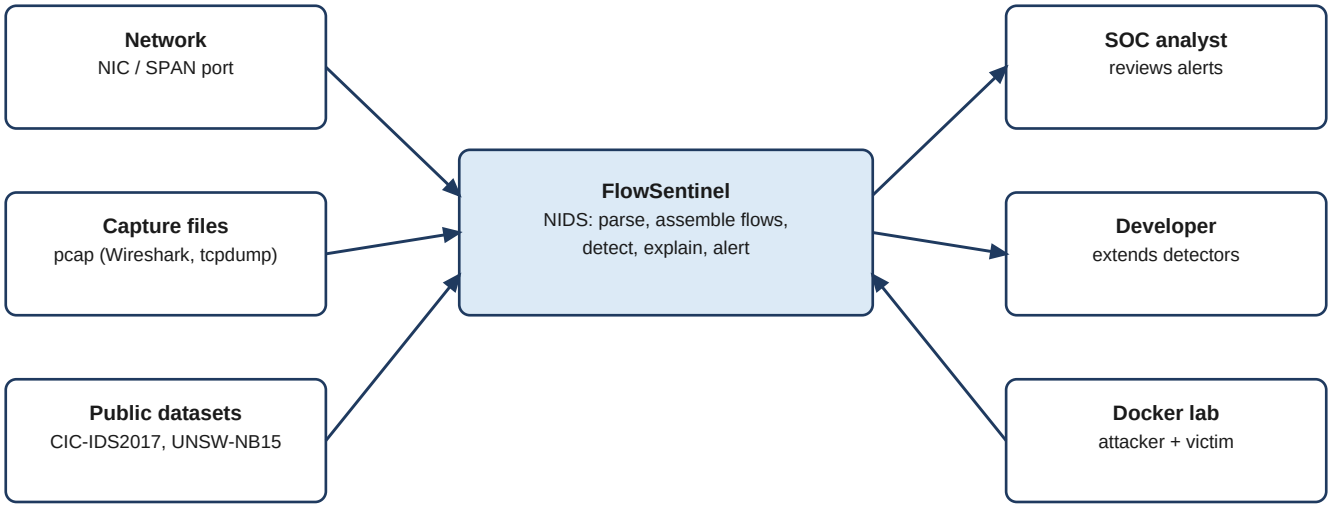


Figure 1. System context: inputs on the left, consumers on the right.

2.2 Product functions (summary)

Function group	What it does	Section
Packet acquisition	Reads pcap files and sniffs live interfaces	5.1
Parsing	Decodes Ethernet, IPv4, TCP, UDP and ICMP into immutable Packet objects	5.2
Flow assembly	Groups packets into bidirectional flows with timeouts and a bounded table	5.3
Feature extraction	Computes per-flow and per-host sliding-window features	5.4
Rule detection	Port scan, SYN flood, brute force and host sweep with tunable thresholds	5.5
ML detection	Isolation Forest and Random Forest with verified model files	5.6
Explainability	Rule reasons with value and threshold; SHAP top features for ML alerts	5.7
Alert management	De-duplication, severity, SQLite storage, export and query	5.8
Interfaces	CLI, REST API and web dashboard	5.9 to 5.11
Configuration, lab, evaluation	YAML configuration, Docker attack lab, metrics and benchmarks	5.12 to 5.14
Monitoring and process	Counters, drift check, logging, CI, packaging	5.15 to 5.17

2.3 User classes and characteristics

User class	Characteristics	Main needs	Priority
Security analyst	Understands networking and attacks; not necessarily ML	Clear alerts with reasons, low false alarms, quick triage	Primary
Security student / researcher	Learns detection; runs lab experiments	Reproducible lab, documented features, honest evaluation	Primary
Developer / contributor	Python developer; may know C++	Clean modules, tests, plug-in detector interface, contributor guide	Secondary
Operator	Installs and runs a sensor on a host	Simple install, safe defaults, stable resource use, logs	Secondary

2.4 Operating environment

Item	Specification
Operating systems	Windows 10/11, Ubuntu 22.04 or later (including Kali Linux), macOS 13 or later (offline analysis)
Runtime	Python 3.10, 3.11 or 3.12; optional C++17 compiler for the parser extension
Capture libraries	Npcap on Windows (installed with Wireshark), libpcap on Linux and macOS; live capture needs administrator or root rights
Reference machine (for performance figures)	4 CPU cores, 8 GB RAM, SSD, 64-bit OS, Python 3.12
Minimum machine	2 CPU cores, 4 GB RAM; offline analysis of captures up to 500 MB
Browser (dashboard)	Current Chrome, Firefox or Edge
Optional tools	Docker 24 or later with Compose for the attack lab; Wireshark for inspecting captures

2.5 Design and implementation constraints

ID	Constraint
CON-01	Implementation language is Python 3.10+; performance-critical parsing may have an optional C++ module with a Python fallback.
CON-02	The project licence is MIT; dependency licences must permit distribution (see NFR-ETH-04, -05).
CON-03	The parser and flow engine use only the Python standard library so the core stays small, auditable and licence-clean.
CON-04	Version 1.0 handles IPv4 over Ethernet and classic pcap only.
CON-05	Detection uses flow metadata only; payload is never inspected or stored by default.
CON-06	Attack tooling runs only inside the isolated Docker lab; real captures committed to the repository must be anonymised.
CON-07	One developer and four months of calendar time; scope is managed with MoSCoW priorities and monthly releases.

2.6 Assumptions and dependencies

ID	Assumption or dependency
A-01	The monitored link carries mostly IPv4 TCP, UDP and ICMP; encrypted traffic still exposes sizes, timing and flags.
A-02	Public datasets (CIC-IDS2017, UNSW-NB15) remain available and their labels are accurate enough for model training.
A-03	Users have the right to capture and analyse the traffic they feed to the system.
A-04	A sensor host can see the traffic it analyses (SPAN port, tap or a host's own interface).
D-01	scapy for live capture and test cross-checks; scikit-learn, XGBoost and SHAP for ML; FastAPI and uvicorn for the API; PyYAML for configuration; pybind11 for the optional C++ module.
D-02	GitHub Actions for CI; Docker for the lab; Wireshark and Npcap for capturing test traffic on Windows.

2.7 Release plan

Release	Window	Theme	Exit criteria
R1 v0.1.0	mid-Oct to mid-Nov 2026	Foundations: capture files, parser, flows, flow features, CLI analyze, CI	Acceptance tests AT-01, AT-02; parser matches scapy on fixture; CI green
R2 v0.2.0	mid-Nov to mid-Dec 2026	Docker lab, host-window features, rule detectors, alerts in SQLite, configuration	AT-03 to AT-07; recall 0.95; benign alert rate 0.1%
R3 v0.3.0	mid-Dec 2026 to mid-Jan 2027	ML models, evaluation, SHAP explanations, hybrid mode	AT-08; evaluation report with confusion matrices and generalisation gap
R4 v1.0.0	mid-Jan to mid-Feb 2027	Live capture, C++ parser, API, dashboard, drift, hardening, documentation	AT-09 to AT-13; coverage 80%; fuzz test passes; README quick start works

2.8 Risk register

ID	Risk	Likelihood	Impact	Mitigation
RK-01	ML models score well on a public dataset but fail on lab traffic (dataset bias)	High	High	Report the generalisation gap; keep transparent rules as the baseline; hybrid mode (FR-ML-09, FR-EVL-02)
RK-02	Too many false alarms make the tool unusable	Medium	High	Tight FPR targets, cooldown suppression, threshold tuning (NFR-ACC-02, FR-DET-08); see Appendix A.6
RK-03	Pure-Python speed is too low for real links	Medium	Medium	Bounded design, benchmarks, optional C++ parser (NFR-PERF-01, -02)
RK-04	Private data leaks into the public repository through captures	Medium	High	Scrub tool, automated privacy test, large-file hook, .gitignore (FR-DEV-05, -06)
RK-05	Loading a malicious model file runs code (pickle)	Low	High	SHA-256 verification, local trusted path only (FR-ML-07, NFR-SEC-02)
RK-06	Scapy's GPL-2.0 licence conflicts with an MIT distribution	Medium	Medium	Keep scapy out of the core; optional extra (NFR-ETH-05)
RK-07	Scope exceeds four months for one developer	High	Medium	MoSCoW priorities; C-items may move to Later; monthly releases
RK-08	Windows capture permissions or Npcap problems block live tests	Medium	Low	Offline mode is complete without live capture; documented setup

3. Use cases

Each use case lists the actor, the preconditions, the main flow and the outcome, with the requirement groups that realise it.

UC-01 Analyse a capture offline

Actor	Analyst or student
Preconditions	Package installed; a classic pcap file exists.
Main flow	1. Run `flowsentinel analyze capture.pcap`. 2. The system reads packets, builds flows and features. 3. Detectors run. 4. The summary lists packets, flows, parse errors and alerts with reasons.
Outcome	Alerts are stored and exported; exit code 0.
Requirements	FR-CAP, PAR, FLW, FEA, DET, EXP, ALR, CLI

UC-02 Monitor live traffic

Actor	Operator
Preconditions	Npcap or libpcap installed; administrator rights; interface name known.
Main flow	1. Run `flowsentinel capture --iface Wi-Fi`. 2. A capture thread feeds a bounded queue. 3. The pipeline raises alerts within 2 s of the evidence. 4. Ctrl+C flushes flows and exits cleanly.
Outcome	Alerts persisted; counters show packets read and dropped.
Requirements	FR-CAP-05..07, FR-MON-01, NFR-PERF-03, NFR-REL-02

UC-03 Investigate an alert

Actor	Analyst
Preconditions	Alerts exist; API and dashboard are running.
Main flow	1. Open the dashboard. 2. Filter by severity. 3. Open an alert. 4. Read the reasons (measured value versus threshold, or top SHAP features) and the flow summary. 5. Decide: ignore, tune a threshold or escalate.
Outcome	The analyst understands why the alert fired without reading code.
Requirements	FR-EXP, ALR, API, UI

UC-04 Train and evaluate a model

Actor	Student or researcher
Preconditions	Dataset downloaded locally; ml extra installed.
Main flow	1. Run `flowsentinel train` with a seed. 2. Run `flowsentinel evaluate` on the public dataset and on lab captures. 3. Read the report: confusion matrix, ROC-AUC, PR-AUC and the generalisation gap. 4. Tune the threshold for a target false positive rate.
Outcome	A versioned, checksummed model and a reproducible report.
Requirements	FR-ML, EVL, FEA-05

UC-05 Reproduce lab experiments

Actor	Student
Preconditions	Docker installed.
Main flow	1. Start the lab with one command. 2. Run the attack scripts (scan, SYN flood, brute force) and the benign script. 3. Collect labelled captures. 4. Analyse them with UC-01 and compare with the labels.
Outcome	Anyone can repeat the published detection results.
Requirements	FR-LAB, EVL, NFR-ETH-02

UC-06 Add a new detector

Actor	Developer
Preconditions	Repository cloned; development tools installed.
Main flow	1. Implement the detector interface. 2. Add configuration keys with defaults. 3. Add unit tests with synthetic frames and lab pcaps. 4. Open a pull request; CI must pass.
Outcome	New detector available without changing core code.
Requirements	FR-DET-07, CFG, DEV, NFR-MNT

4. System architecture

4.1 Logical architecture

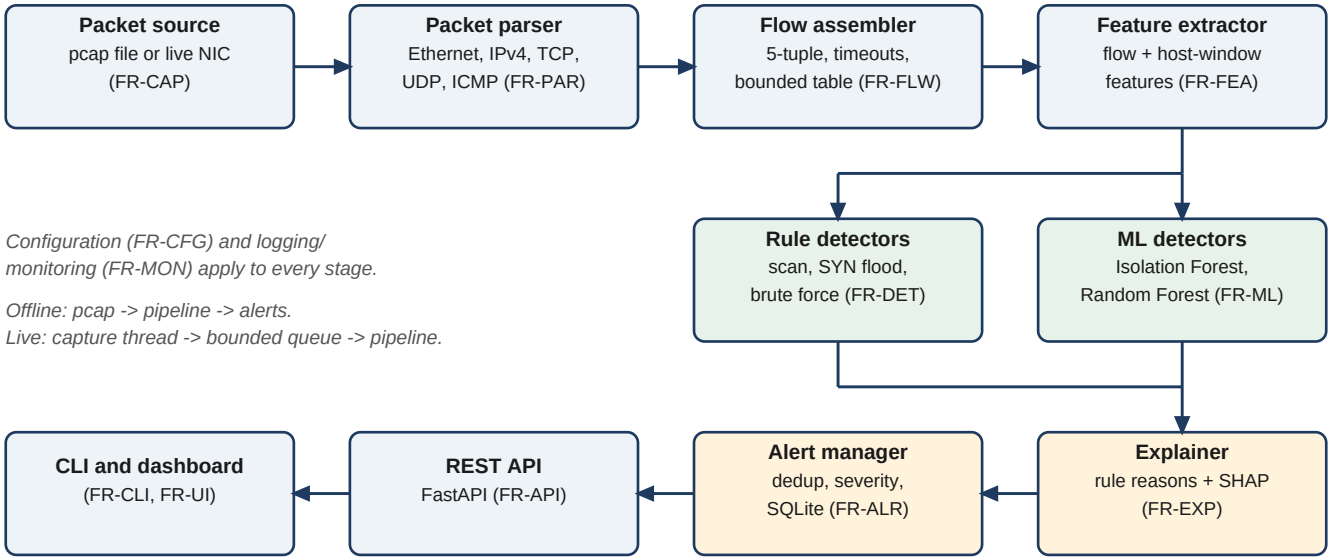


Figure 2. Processing pipeline. Green boxes are detectors, amber boxes produce the final alert.

4.2 Components

Component	Responsibility	Requirements
Packet source	Yields raw frames with timestamps from a pcap file or a live interface	FR-CAP
Parser	Turns frames into immutable Packet objects; never raises on non-IPv4	FR-PAR
Flow assembler	Maintains the bounded flow table; emits finished flows	FR-FLW
Feature extractor	Computes flow features and per-host window features	FR-FEA
Rule detectors	Evaluate thresholds on host-window features; common detector interface	FR-DET
ML detectors	Score flow features with verified models	FR-ML
Explainer	Builds reasons from rules or SHAP values	FR-EXP
Alert manager	De-duplicates, assigns severity, stores, exports and queries alerts	FR-ALR
Interfaces	CLI, FastAPI service, dashboard	FR-CLI, API, UI
Configuration, logging, monitoring	Cross-cutting: settings, counters, structured logs, drift	FR-CFG, MON
Lab and evaluation tools	Docker lab, dataset loaders, metrics, benchmarks	FR-LAB, EVL

4.3 Repository layout (target)

```
flowsentinel/
  src/flowsentinel/
    parser/        packet.py frames.py pcap.py          (implemented)
    flows/         flow table, timeouts, flow ids
    features/      flow features, host-window features
    detectors/     rule detectors, ML detectors, hybrid
    explain/       reason templates, SHAP wrapper
    alerts/        alert model, SQLite store, export
    api/           FastAPI app          dashboard/  static UI
    cli.py  config.py  logging.py
  scripts/         scrub_pcap.py, dataset loaders, benchmarks
  lab/             docker-compose.yml, attack and benign scripts
  tests/           unit tests, fixtures/sample_web.pcap
  docs/            parser.md, architecture, ADRs
  .github/workflows/ci.yml   .pre-commit-config.yaml   pyproject.toml
```

4.4 Data flow

- 1. The packet source yields (timestamp, frame) pairs.
- 2. The parser returns a Packet or None (non-IPv4); malformed frames are counted and skipped.
- 3. The flow assembler updates or creates a flow for the canonical 5-tuple and emits flows that time out, finish or are evicted.
- 4. The feature extractor updates per-host windows on each packet and computes flow features when a flow is emitted.
- 5. Rule detectors evaluate host-window features continuously; ML detectors score each emitted flow.
- 6. The explainer attaches reasons; the alert manager suppresses duplicates, sets severity and stores the alert.
- 7. The CLI prints, the API serves and the dashboard displays stored alerts.

4.5 Live-mode concurrency

In live mode a **capture thread** reads frames and puts them on a bounded queue. A **processing thread** runs steps 2 to 6. A separate **service thread** runs the API and dashboard against the alert database only. If the queue is full the capture thread drops the packet and increments a counter, so a slow detector degrades gracefully instead of exhausting memory (FR-CAP-07, NFR-SCL-01, NFR-REL-05).

5. Functional requirements

Each group below states its purpose and then lists its requirements. The group code gives the prefix of the IDs (for example `FR-PAR-05`). The status column reflects the repository on the document date; the parser requirements are implemented and covered by 42 automated tests, including a cross-check against scapy.

5.1 Packet acquisition (FR-CAP)

Reads packets from capture files (offline mode) and from network interfaces (live mode).

ID	Requirement	Pri	Ver	Rel	Status
FR-CAP-01	The system shall read packets from classic libpcap files in either byte order, with microsecond or nanosecond timestamp resolution.	M	T	R1	**Implemented**
FR-CAP-02	The system shall reject files that are not classic pcap (including pcapng) with an error that names the problem and tells the user how to fix it.	M	T	R1	**Implemented**
FR-CAP-03	The system shall accept only the Ethernet link type (1) and shall report any other link type by number.	M	T	R1	**Implemented**
FR-CAP-04	The system shall tolerate a truncated final record by logging a warning and returning every complete packet read before it.	M	T	R1	**Implemented**
FR-CAP-05	The system shall capture packets live from a user-selected network interface, optionally restricted by a capture filter expression.	M	D	R4	**Planned**
FR-CAP-06	The system shall list the capture interfaces available on the host.	S	D	R4	**Planned**
FR-CAP-07	In live mode the capture thread shall pass packets to the processing thread through a bounded queue and shall count packets dropped when the queue is full.	S	T	R4	**Planned**
FR-CAP-08	The system could read pcapng files directly in addition to classic pcap.	C	T	Later	**Planned**
FR-CAP-09	The pcap reader shall reject any record whose captured length exceeds 262,144 bytes instead of allocating it.	M	T	R4	**Planned**

5.2 Packet parsing (FR-PAR)

Converts raw Ethernet frames into typed, immutable Packet objects. The parser uses only the Python standard library.

ID	Requirement	Pri	Ver	Rel	Status
FR-PAR-01	The parser shall decode Ethernet II frames and shall skip one 802.1Q VLAN tag when present.	M	T	R1	**Implemented**
FR-PAR-02	The parser shall decode the IPv4 header: version, header length (including options), total length, TTL, protocol, source and destination address.	M	T	R1	**Implemented**
FR-PAR-03	The parser shall decode TCP: ports, sequence number, acknowledgement number, window, flag byte and data offset (header options skipped).	M	T	R1	**Implemented**
FR-PAR-04	The parser shall decode UDP ports and shall compute ICMP payload length after the 8-byte ICMP header.	M	T	R1	**Implemented**
FR-PAR-05	Payload length shall be computed from the IPv4 total length minus the IP and transport headers, never from the frame size, so Ethernet padding is not counted as payload.	M	T	R1	**Implemented**
FR-PAR-06	When the IPv4 total length is 0 (segmentation offload on the capturing host), the parser shall use the captured length instead.	M	T	R1	**Implemented**
FR-PAR-07	For non-first IP fragments the parser shall leave ports and TCP fields empty because no transport header is present.	M	T	R1	**Implemented**
FR-PAR-08	Frames that are not IPv4 shall be skipped silently; malformed IPv4 frames shall raise ParseError; the pcap reader shall log a warning for each and continue.	M	T	R1	**Implemented**

ID	Requirement	Pri	Ver	Rel	Status
FR-PAR-09	Parsing shall be deterministic and free of side effects; the Packet value object shall be immutable.	M	T	R1	Implemented
FR-PAR-10	The parser should decode IPv6 headers and extension headers.	S	T	Later	Planned
FR-PAR-11	The system shall provide an optional C++ implementation of the frame parser (pybind11) returning results identical to the Python parser, with automatic fallback to Python.	S	T	R4	Planned

5.3 Flow assembly (FR-FLW)

Groups packets into bidirectional flows so that detectors reason about conversations, not single packets.

ID	Requirement	Pri	Ver	Rel	Status
FR-FLW-01	The system shall group packets into bidirectional flows keyed by a canonical 5-tuple (protocol and the two endpoints in a fixed order) so both directions map to one flow.	M	T	R1	Planned
FR-FLW-02	Each flow shall record its initiator (source of the first packet) and label every packet forward or backward relative to it.	M	T	R1	Planned
FR-FLW-03	A flow shall be closed after an idle timeout (default 60 s) or an active timeout (default 120 s); both shall be configurable.	M	T	R1	Planned
FR-FLW-04	A TCP flow shall also be closed when a FIN has been seen in both directions or when an RST is seen.	M	T	R1	Planned
FR-FLW-05	All flows still open at the end of the input shall be emitted (flushed).	M	T	R1	Planned
FR-FLW-06	The flow table shall be bounded by a configurable maximum of active flows (default 100,000); when full, the least recently active flow shall be evicted, emitted with reason 'evicted', and counted.	M	T	R2	Planned
FR-FLW-07	Each flow shall receive a deterministic identifier derived from its key and start time, so repeated runs on the same input produce the same identifiers.	S	T	R1	Planned
FR-FLW-08	The flow engine shall not let its clock move backwards when packet timestamps are out of order, and shall count such events.	S	T	R2	Planned
FR-FLW-09	UDP and ICMP flows shall be supported using timeouts only.	M	T	R1	Planned

5.4 Feature extraction (FR-FEA)

Turns flows and recent host behaviour into numeric features for rule and ML detectors.

ID	Requirement	Pri	Ver	Rel	Status
FR-FEA-01	The system shall compute per-flow features: duration; packet and byte counts forward and backward; packets per second; bytes per second; packet size min/mean/max/standard deviation; inter-arrival time min/mean/max/standard deviation.	M	T	R1	Planned
FR-FEA-02	The system shall compute per-flow TCP features: counts of SYN, FIN, RST, PSH, ACK and URG flags; initial window size in each direction; number of packets carrying payload.	M	T	R1	Planned
FR-FEA-03	The system shall compute per-source-host features over a sliding window (default 10 s): distinct destination ports, distinct destination hosts, new-connection rate, SYN-only ratio, failed-connection ratio and Shannon entropy of destination ports in bits.	M	T	R2	Planned
FR-FEA-04	Feature values shall be deterministic; undefined values (for example division by zero) shall take a documented default and shall never be NaN or infinite.	M	T	R1	Planned
FR-FEA-05	The feature vector shall have a documented, versioned schema (name, type, unit), and the schema version shall be stored with every trained model.	M	I	R3	Planned
FR-FEA-06	The system shall export features as a pandas DataFrame and as CSV.	S	T	R1	Planned
FR-FEA-07	Sliding-window state shall be bounded by discarding entries older than the window.	M	T	R2	Planned

5.5 Rule-based detection (FR-DET)

Transparent detectors with configurable thresholds. Defaults are starting points to be tuned on lab data.

ID	Requirement	Pri	Ver	Rel	Status
FR-DET-01	Port-scan detector: raise an alert when one source contacts at least N distinct destination ports (default 50) within window W (default 10 s) and the destination-port entropy is at least H bits (default 4.0).	M	T	R2	**Planned**
FR-DET-02	SYN-flood detector: raise an alert when SYN-only packets toward one destination exceed R per second (default 200) and fewer than 10% of those attempts complete the handshake.	M	T	R2	**Planned**
FR-DET-03	Brute-force detector: raise an alert when one source opens at least N new TCP connections (default 20) to the same destination and service port within 60 s; monitored ports default to 21, 22, 23 and 3389.	M	T	R2	**Planned**
FR-DET-04	Host-sweep detector: raise an alert when one source contacts at least N distinct hosts (default 20) within the window.	S	T	R2	**Planned**
FR-DET-05	All thresholds shall be configurable with documented defaults; invalid values shall be rejected at start-up with a message naming the key and the allowed range.	M	T	R2	**Planned**
FR-DET-06	Each detector shall be individually switchable on or off.	M	T	R2	**Planned**
FR-DET-07	Detectors shall implement a common interface so a new detector can be added without changing the core pipeline.	S	I	R2	**Planned**
FR-DET-08	Repeated alerts with the same type, source and target shall be suppressed within a cooldown (default 60 s) while a counter on the original alert is incremented.	M	T	R2	**Planned**

5.6 Machine-learning detection (FR-ML)

Supervised and unsupervised models trained on flow features only; never on payload.

ID	Requirement	Pri	Ver	Rel	Status
FR-ML-01	The system shall train an Isolation Forest on benign flow features for unsupervised anomaly detection.	M	T	R3	**Planned**
FR-ML-02	The system shall train a supervised classifier (Random Forest, optionally gradient boosting) on labelled flows.	M	T	R3	**Planned**
FR-ML-03	Training shall be reproducible: fixed random seed, recorded library versions and a hash of the training data.	M	T	R3	**Planned**
FR-ML-04	Class imbalance shall be handled by class weights or resampling; the method shall be configurable and recorded in the model metadata.	M	I	R3	**Planned**
FR-ML-05	Evaluation splits shall be time-aware or attack-type-aware to avoid leakage; results from a purely random split shall not be the only reported figure.	M	I	R3	**Planned**
FR-ML-06	A saved model shall carry metadata: feature schema version, training data hash, metrics, creation time and the SHA-256 of the model file.	M	T	R3	**Planned**
FR-ML-07	Loading a model shall verify its SHA-256 against the metadata and shall refuse to load on mismatch, because pickle-based formats can execute code.	M	T	R3	**Planned**
FR-ML-08	The detection threshold shall be configurable and tunable on a validation set to meet a target false positive rate.	M	T	R3	**Planned**
FR-ML-09	Hybrid mode shall raise an alert when any rule fires or the ML score exceeds the threshold, and the alert shall record which source triggered it.	M	T	R3	**Planned**
FR-ML-10	The ML detector shall score flows from the feature vector only and shall never read payload bytes.	M	I	R3	**Planned**

5.7 Explainability (FR-EXP)

Every alert must tell the analyst why it was raised, in plain language.

ID	Requirement	Pri	Ver	Rel	Status
FR-EXP-01	Every alert shall contain at least one human-readable reason.	M	T	R2	Planned
FR-EXP-02	Rule alerts shall state the measured value and the threshold, for example 'unique destination ports = 100 (threshold >= 50) in 10 s'.	M	T	R2	Planned
FR-EXP-03	ML alerts shall list the top k contributing features (default 5) with direction and magnitude, computed with SHAP values.	M	T	R3	Planned
FR-EXP-04	Reason text shall be produced from templates that map feature names to plain-language names and avoid ML jargon.	S	I	R3	Planned
FR-EXP-05	Explanation computation shall add no more than 500 ms per alert in live mode, using lazy or asynchronous computation if needed.	S	T	R4	Planned
FR-EXP-06	Explanations shall be stored together with the alert.	M	T	R3	Planned

5.8 Alert management and storage (FR-ALR)

Creates, de-duplicates, stores and exports alerts.

ID	Requirement	Pri	Ver	Rel	Status
FR-ALR-01	An alert shall contain: id, timestamp, type, severity, score, source IP, destination IP, destination port, protocol, flow id, detector (rule, ML or hybrid), reasons, suppressed count and model version.	M	T	R2	Planned
FR-ALR-02	Severity shall be one of low, medium, high or critical with a documented mapping from rule or score.	M	I	R2	Planned
FR-ALR-03	Alerts shall be persisted in SQLite (write-ahead logging) with a schema version and shall survive process restart.	M	T	R2	Planned
FR-ALR-04	Alerts shall be exportable as JSON Lines and CSV.	M	T	R2	Planned
FR-ALR-05	Alerts shall be queryable by time range, type, severity, source and destination.	M	T	R2	Planned
FR-ALR-06	Alerts older than a configurable retention period (default 30 days) shall be purged at start-up.	S	T	R2	Planned
FR-ALR-07	The exported alert JSON may map to Elastic Common Schema field names.	C	I	Later	Planned

5.9 Command-line interface (FR-CLI)

The primary way to run the system.

ID	Requirement	Pri	Ver	Rel	Status
FR-CLI-01	'flowsentinel analyze <pcap>' shall run the full pipeline offline, print a summary, and exit with 0 on success, 1 on runtime error and 2 on invalid arguments.	M	D	R1	Planned
FR-CLI-02	'flowsentinel capture --iface <name>' shall run live detection and shut down cleanly on Ctrl+C.	M	D	R4	Planned
FR-CLI-03	'flowsentinel train' and 'flowsentinel evaluate' shall train and evaluate models and write reports.	M	D	R3	Planned
FR-CLI-04	'flowsentinel serve' shall start the API and dashboard.	M	D	R4	Planned
FR-CLI-05	'flowsentinel config validate <file>' shall validate a configuration file and list every error.	S	D	R2	Planned
FR-CLI-06	Every command shall support --help, --version, --format (table or json) and --log-level.	M	T	R1	Planned
FR-CLI-07	Errors shall go to standard error as actionable text; stack traces shall appear only with --debug.	S	T	R1	Planned

5.10 REST API (FR-API)

Programmatic access to alerts, flows and statistics.

ID	Requirement	Pri	Ver	Rel	Status
FR-API-01	The system shall provide a FastAPI service with GET /health, GET /alerts, GET /alerts/{id}, GET /flows/{id} and GET /stats.	M	T	R4	**Planned**
FR-API-02	List endpoints shall support pagination (limit default 50, maximum 500, and offset) and filters for time range, type, severity and source.	M	T	R4	**Planned**
FR-API-03	The API shall publish an OpenAPI description at /docs.	S	I	R4	**Planned**
FR-API-04	The API shall bind to 127.0.0.1 by default; binding to other addresses shall require an explicit flag and an API token.	M	T	R4	**Planned**
FR-API-05	When a token is configured, the API shall require it as a bearer token and compare it in constant time.	S	T	R4	**Planned**
FR-API-06	Invalid input shall produce a JSON error body with HTTP status 400, 404 or 422; unexpected failures shall produce 500 without internal details.	M	T	R4	**Planned**
FR-API-07	The API could accept a pcap upload (default size limit 50 MB) for offline analysis.	C	T	Later	**Planned**

5.11 Dashboard (FR-UI)

A simple local web view for analysts.

ID	Requirement	Pri	Ver	Rel	Status
FR-UI-01	The dashboard shall show a live alert table, newest first, with severity colour and filters by type and severity.	M	D	R4	**Planned**
FR-UI-02	The dashboard shall chart alerts over time by type and list top talkers by bytes and by flows.	S	D	R4	**Planned**
FR-UI-03	Selecting an alert shall show its reasons, key feature values and a flow summary.	M	D	R4	**Planned**
FR-UI-04	The dashboard shall refresh automatically every 5 seconds (configurable) and work in current Chrome, Firefox and Edge.	S	D	R4	**Planned**
FR-UI-05	The dashboard shall work without any external network request (no CDN assets).	S	I	R4	**Planned**

5.12 Configuration (FR-CFG)

One predictable way to tune the system.

ID	Requirement	Pri	Ver	Rel	Status
FR-CFG-01	The system shall read a YAML configuration file given by --config and shall have built-in defaults for every key.	M	T	R2	**Planned**
FR-CFG-02	Precedence shall be: command-line flags, then environment variables (prefix FLOWSENTINEL_), then the file, then defaults.	M	T	R2	**Planned**
FR-CFG-03	The configuration shall be validated against a schema and every error shall name the key and the allowed range.	M	T	R2	**Planned**
FR-CFG-04	Configuration shall be parsed with a safe loader that cannot execute code.	M	T	R2	**Planned**

5.13 Attack lab (FR-LAB)

A reproducible, isolated environment that produces labelled traffic.

ID	Requirement	Pri	Ver	Rel	Status
FR-LAB-01	The repository shall include a Docker Compose lab with an isolated network containing a victim (web server and SSH), an attacker (Nmap, hping3, Hydra) and a capture point.	M	D	R2	Planned
FR-LAB-02	Scripts shall generate labelled captures for port scans, SYN floods, brute force and benign browsing.	M	D	R2	Planned
FR-LAB-03	The lab network shall have no route to the internet or the host LAN.	M	I	R2	Planned
FR-LAB-04	Each capture shall have a label file (CSV) giving attacker address, attack type and time range.	M	I	R2	Planned
FR-LAB-05	The lab shall start and stop with one documented command.	S	D	R2	Planned

5.14 Evaluation and benchmarking (FR-EVL)

Honest measurement of detection quality and speed.

ID	Requirement	Pri	Ver	Rel	Status
FR-EVL-01	The evaluation tool shall compute precision, recall, F1, false positive rate, ROC-AUC, PR-AUC and the confusion matrix, overall and per attack class.	M	T	R3	Planned
FR-EVL-02	Evaluation shall cover at least one public dataset (CIC-IDS2017 or UNSW-NB15) and the lab traffic, and shall report the generalisation gap between them.	M	A	R3	Planned
FR-EVL-03	Reports shall be written as Markdown and JSON including seed, library versions and configuration.	S	I	R3	Planned
FR-EVL-04	The benchmark tool shall report packets per second per pipeline stage and peak memory for the Python and C++ paths.	M	T	R4	Planned
FR-EVL-05	The evaluation tool shall save a threshold sweep (precision-recall curve data) as CSV and an image.	S	I	R3	Planned

5.15 Monitoring and logging (FR-MON)

Visibility into the health of the sensor itself.

ID	Requirement	Pri	Ver	Rel	Status
FR-MON-01	The system shall maintain counters for packets read, parse errors, packets dropped, active flows, evictions and alerts by type, shown in the CLI summary and at GET /stats.	M	T	R2	Planned
FR-MON-02	The system shall compute the Population Stability Index (PSI) of key features between training data and live traffic per 10-minute window and warn when PSI exceeds 0.25.	S	T	R4	Planned
FR-MON-03	Logging shall be structured, support a JSON option and a level setting, and shall never include payload bytes, credentials or tokens.	M	I	R1	Planned

5.16 Engineering process requirements (FR-DEV)

Requirements on how the project is built, because quality of process is part of the deliverable.

ID	Requirement	Pri	Ver	Rel	Status
FR-DEV-01	The source shall be published in a public GitHub repository under the MIT licence.	M	I	R1	Implemented
FR-DEV-02	Continuous integration shall run ruff check, ruff format --check and pytest on Python 3.10, 3.11 and 3.12 for every push to main and every pull request.	M	T	R1	In progress

ID	Requirement	Pri	Ver	Rel	Status
FR-DEV-03	Pre-commit hooks shall check whitespace, end-of-file, YAML and TOML validity, private keys, files larger than 1 MB, and ruff lint and format.	M	T	R1	In progress
FR-DEV-04	Commits shall follow Conventional Commits; each feature shall be developed on its own branch and merged by pull request using rebase-and-merge.	S	I	R1	In progress
FR-DEV-05	Test captures committed to the repository shall be anonymised (MAC and private IPv4 addresses remapped), smaller than 1 MB, and verified by an automated test.	M	T	R1	In progress
FR-DEV-06	Raw captures shall never be committed; they shall be excluded by .gitignore and by the large-file hook.	M	I	R1	Implemented
FR-DEV-07	Line coverage of the source package shall be at least 80%, enforced in CI.	M	T	R4	Planned
FR-DEV-08	Dependabot alerts shall be enabled and a SECURITY.md shall describe how to report vulnerabilities.	S	I	R2	Planned
FR-DEV-09	Releases shall use Semantic Versioning with annotated tags and a CHANGELOG.	S	I	R1	Planned
FR-DEV-10	Significant design decisions shall be recorded as Architecture Decision Records under docs/adr.	S	I	R1	Planned

5.17 Packaging and distribution (FR-PKG)

How users obtain and install the software.

ID	Requirement	Pri	Ver	Rel	Status
FR-PKG-01	The package shall install with pip from the repository and shall provide the 'flowsentinel' command.	M	D	R4	Planned
FR-PKG-02	Optional dependency groups shall separate capture (scapy), ml (scikit-learn, XGBoost, SHAP), api (FastAPI, uvicorn) and dev tools, so the core installs without them.	S	I	R4	Planned
FR-PKG-03	The package could be published on PyPI.	C	D	Later	Planned

6. External interface requirements

6.1 User interfaces

Command-line interface

Command	Purpose	Key options	Exit codes
analyze PCAP	Offline analysis	--config, --format, --output, --model	0 ok, 1 error, 2 usage
capture	Live detection	--iface, --filter, --config	0, 1, 2
train	Train models	--dataset, --seed, --out	0, 1, 2
evaluate	Metrics and reports	--model, --dataset, --report-dir	0, 1, 2
serve	API and dashboard	--host, --port, --token	0, 1, 2
config validate FILE	Check a configuration file	none	0 valid, 1 invalid

Global options on every command: `--help`, `--version`, `--format table|json`, `--log-level`, `--debug`.

Dashboard

One page with four regions: (1) summary tiles (packets read, flows, alerts, dropped), (2) alert table with severity colour and filters, (3) charts for alerts over time by type and for top talkers, (4) a detail panel showing reasons, key feature values and a flow summary. The page polls the REST API; it loads no external scripts or fonts.

6.2 Hardware interfaces

The system uses a standard network interface through the operating system's capture library. For live monitoring of other hosts, the interface should be connected to a switch SPAN (mirror) port or a network tap. The interface may be placed in promiscuous mode by the capture library. No special hardware is required.

6.3 Software interfaces

Component	Purpose	Interface	Licence (verify at release)
scapy	Live capture, cross-check tests	Python import (optional extra)	GPL-2.0
libpcap / Npcap	Packet capture	Via scapy	BSD / Npcap licence
scikit-learn	Isolation Forest, Random Forest	Python import (ml extra)	BSD-3-Clause
XGBoost	Gradient boosting (optional)	Python import (ml extra)	Apache-2.0
SHAP	Per-alert feature contributions	Python import (ml extra)	MIT
pandas, NumPy	Feature tables	Python import	BSD-3-Clause
FastAPI, uvicorn	REST API	HTTP (api extra)	MIT, BSD-3-Clause
PyYAML	Configuration	Safe loader only	MIT
SQLite	Alert storage	Python sqlite3, WAL mode	Public domain
pybind11	Optional C++ parser binding	Build-time	BSD-3-Clause
Docker Compose	Attack lab	docker compose up	Apache-2.0

6.4 Communication interfaces

The API listens on `127.0.0.1:8000` by default over HTTP with JSON bodies. The system makes no outbound network connections during analysis or detection. All times are UTC in ISO 8601.

Method and path	Description	Parameters	Success response
GET /health	Liveness check	none	200 {status, version}
GET /alerts	List alerts, newest first	limit, offset, since, until, type, severity, src_ip	200 {items[], total, limit, offset}
GET /alerts/{id}	One alert with reasons	id	200 alert; 404 if absent
GET /flows/{id}	Flow summary and features	id	200 flow; 404 if absent
GET /stats	Counters	none	200 {packets_read, parse_errors, dropped, active_flows, evictions, alerts_by_type}

Error body for 400, 404 and 422: `{"error": "message", "field": "name"}`. A 500 response never includes stack traces.

6.5 File formats

Format	Direction	Notes
Classic pcap (.pcap)	Input and test fixtures	Both byte orders; microsecond and nanosecond; Ethernet only; record length bounded (FR-CAP-09)
YAML configuration	Input	Safe loader; see Appendix B
JSON Lines / CSV	Output (alerts, features)	UTF-8, one object per line for JSON Lines
SQLite database	Storage	Schema versioned; WAL mode
joblib model + JSON metadata	Input and output	SHA-256 verified before load
Markdown and JSON reports	Output	Evaluation and benchmark reports

7. Data requirements

7.1 Packet (implemented)

Field	Type	Description
timestamp	float (seconds)	Capture time since the epoch; accurate to about a microsecond for current dates
src_ip, dst_ip	str	Dotted-decimal IPv4 addresses
protocol	int	IP protocol number (1 ICMP, 6 TCP, 17 UDP)
ttl	int	IPv4 time to live
ip_length	int	IPv4 total length in bytes (captured length if recorded as 0)
payload_len	int	Bytes after the transport header, from IP lengths (never from frame size)
src_port, dst_port	int or None	TCP/UDP ports; None for ICMP, other protocols and non-first fragments
tcp_flags	TcpFlags	FIN 0x01, SYN 0x02, RST 0x04, PSH 0x08, ACK 0x10, URG 0x20, ECE 0x40, CWR 0x80
seq, ack, window	int or None	TCP header values; None when not TCP

7.2 Flow record

Field	Type	Description
flow_id	str	Deterministic hash of the key and start time
key	(protocol, endpoint A, endpoint B)	Canonical order: lower address (then port) first, so both directions match
initiator	endpoint	Source of the first packet; defines the forward direction
start_ts, end_ts	float	First and last packet times
end_reason	enum	fin, rst, idle, active, evicted, eof
counters	ints	Packets and bytes forward/backward, flag counts, initial windows
sizes, iats	running statistics	Min, mean, max, standard deviation without storing every packet

7.3 Feature schema version 1

Flow features use prefix `f_`; host-window features use prefix `h_`. Undefined values take 0 (FR-FEA-04).

Feature	Unit	Definition
f_duration_s	seconds	end_ts - start_ts
f_pkts_fwd, f_pkts_bwd	count	Packets in each direction
f_bytes_fwd, f_bytes_bwd	bytes	IP bytes in each direction
f_pps, f_bps	1/s	Total packets or bytes divided by duration (0 when duration is 0)
f_len_min/mean/max/std	bytes	Packet length statistics
f_iat_min/mean/max/std	seconds	Inter-arrival time statistics
f_syn, f_fin, f_rst, f_psh, f_ack, f_urg	count	TCP flag counts
f_win_fwd, f_win_bwd	bytes	First TCP window value seen in each direction
f_payload_pkts	count	Packets with payload_len above 0
h_unique_dst_ports	count	Distinct destination ports contacted by the source in the window
h_unique_dst_hosts	count	Distinct destination addresses in the window
h_new_conn_rate	1/s	New connections per second in the window
h_syn_only_ratio	0 to 1	SYN packets without ACK divided by all TCP packets from the source
h_failed_ratio	0 to 1	Connections ended by RST or unanswered divided by connections
h_port_entropy_bits	bits	Shannon entropy of destination ports (Appendix A.2)

7.4 Alert record and database

Column	Type	Notes
id	INTEGER PRIMARY KEY	Auto-increment
ts	TEXT (ISO 8601 UTC)	Time of detection
type	TEXT	port_scan, syn_flood, brute_force, host_sweep, ml_anomaly, ml_attack
severity	TEXT	low, medium, high, critical
detector	TEXT	rule, ml or hybrid
score	REAL, nullable	ML score between 0 and 1
src_ip, dst_ip, dst_port, protocol	TEXT/INTEGER	Endpoints involved
flow_id	TEXT, nullable	Link to the flow when the alert is flow-based
reasons	TEXT (JSON array)	Plain-language reasons (FR-EXP)
suppressed_count	INTEGER	Duplicates suppressed within the cooldown
model_version	TEXT, nullable	Model id when ML contributed
schema_version	table meta	Integer stored once; checked at start-up

7.5 Model metadata

Each model file has a JSON sidecar with: `model_id`, `algorithm`, `created_utc`, `seed`, `feature_schema_version`, `training_data_sha256`, `library_versions`, `imbalance_method`, `threshold`, `metrics` and `model_sha256` (FR-ML-06, -07).

7.6 Retention, privacy and integrity

- Alerts are kept 30 days by default (FR-ALR-06). Raw captures are never stored by the system.
- Only metadata and features are persisted; payload bytes are not (NFR-SEC-05).
- Committed captures are anonymised: MAC addresses become 02:00:00:00:00:xx and private IPv4 addresses become 10.0.0.x (FR-DEV-05).
- The database is the only shared state between the processing thread and the service thread; writes are transactional (NFR-REL-03).

8. Non-functional requirements

Quality attributes are numbered `NFR-GROUP-NN` and use the same columns as the functional requirements. Numerical targets are justified in Appendix A and will be re-checked against measurements at release R3 and R4.

8.1 Performance efficiency (NFR-PERF)

ID	Requirement	Pri	Ver	Rel	Status
NFR-PERF-01	In offline mode with the pure-Python parser, the pipeline (parse, flow assembly, features, rules) shall sustain at least 20,000 packets per second on the reference machine (Appendix A.5).	M	T	R4	**Planned**
NFR-PERF-02	With the optional C++ parser, the parse stage shall be at least 5 times faster than the pure-Python parser on the same input.	S	T	R4	**Planned**
NFR-PERF-03	In live mode, rule-based alerts shall be raised within 2 seconds of the packet that completes the evidence.	M	T	R4	**Planned**
NFR-PERF-04	Resident memory shall not exceed 512 MB with 100,000 active flows.	M	T	R4	**Planned**
NFR-PERF-05	The ML stage shall score at least 5,000 flows per second in batch mode.	S	T	R3	**Planned**
NFR-PERF-06	The dashboard shall render the alert table within 2 seconds with 10,000 stored alerts.	S	T	R4	**Planned**
NFR-PERF-07	'flowsentinel --help' shall start and print within 2 seconds.	C	T	R4	**Planned**

8.2 Reliability (NFR-REL)

ID	Requirement	Pri	Ver	Rel	Status
NFR-REL-01	The system shall not terminate on malformed packets: 10,000 randomly generated and mutated frames shall be processed without an unhandled exception (fuzz test).	M	T	R4	**Planned**
NFR-REL-02	On Ctrl+C or SIGINT the system shall flush active flows, persist pending alerts and exit within 5 seconds.	M	T	R4	**Planned**
NFR-REL-03	Each alert shall be written atomically; a crash shall not corrupt the alert database.	M	T	R2	**Planned**
NFR-REL-04	The same input, configuration and model shall produce identical alerts (apart from processing timestamps).	M	T	R2	**Planned**
NFR-REL-05	A failure of the API or dashboard shall not stop capture or detection.	S	T	R4	**Planned**

8.3 Detection quality (NFR-ACC)

ID	Requirement	Pri	Ver	Rel	Status
NFR-ACC-01	With default thresholds, rule detectors shall reach recall of at least 0.95 on lab attack traffic (scan, SYN flood, brute force).	M	T	R2	**Planned**
NFR-ACC-02	With default thresholds, no more than 0.1% of benign source-host windows in lab traffic shall raise a rule alert.	M	T	R2	**Planned**
NFR-ACC-03	The supervised ML detector shall reach macro-averaged F1 of at least 0.85 on a time-aware held-out split of the public dataset; cross-dataset results shall be reported even when lower.	S	T	R3	**Planned**
NFR-ACC-04	Every reported result shall include the confusion matrix and not accuracy alone.	M	I	R3	**Planned**
NFR-ACC-05	For the project fixture, parser output shall match an independent reference implementation (scapy) on every compared field.	M	T	R1	**Implemented**

8.4 Security (NFR-SEC)

ID	Requirement	Pri	Ver	Rel	Status
NFR-SEC-01	Configuration and rule files shall be loaded with safe parsers that cannot execute code.	M	T	R2	Planned
NFR-SEC-02	Model files shall be loaded only from a user-specified local path and only after SHA-256 verification.	M	T	R3	Planned
NFR-SEC-03	The API shall listen on loopback by default; remote access shall require an explicit flag and a token.	M	T	R4	Planned
NFR-SEC-04	All external input (capture files, API parameters, configuration) shall be validated for type, range and size.	M	T	R4	Planned
NFR-SEC-05	Payload bytes shall not be persisted by default; only metadata and features shall be stored.	M	I	R2	Planned
NFR-SEC-06	Logs shall not contain payload data, credentials or tokens.	M	I	R1	Planned
NFR-SEC-07	Test fixtures and documentation shall contain no real personal network identifiers, verified by an automated test.	M	T	R1	In progress
NFR-SEC-08	Offline analysis shall run without elevated privileges; elevated privileges shall be required only for live capture and shall be documented.	M	I	R4	Planned

8.5 Ethics and legal (NFR-ETH)

ID	Requirement	Pri	Ver	Rel	Status
NFR-ETH-01	Documentation shall state that the system is defensive and may be used only on networks the user owns or is authorised to monitor.	M	I	R1	Planned
NFR-ETH-02	Attack tooling shall be confined to the isolated lab; no script shall target external hosts.	M	I	R2	Planned
NFR-ETH-03	Public datasets shall be cited with their licences and shall not be redistributed in the repository.	M	I	R3	Planned
NFR-ETH-04	The project licence is MIT; every dependency licence shall be reviewed for compatibility before each release.	M	I	R4	Planned
NFR-ETH-05	Scapy is licensed GPL-2.0; the core library (parser, flows, features, detectors) shall not import it, and it shall be confined to the optional capture extra and test tooling.	S	I	R4	Planned

8.6 Usability (NFR-USE)

ID	Requirement	Pri	Ver	Rel	Status
NFR-USE-01	A new user shall be able to install the package and analyse a capture in no more than five commands documented in the README quick start.	M	D	R4	Planned
NFR-USE-02	Every command shall offer --help with examples; error messages shall state the cause and a remedy.	S	I	R1	Planned
NFR-USE-03	Alert text shall be understandable by a reader with networking knowledge and no machine-learning knowledge (reviewed by at least three readers).	S	I	R3	Planned
NFR-USE-04	The dashboard shall be usable at widths of 1024 px and above and by keyboard navigation.	C	D	R4	Planned

8.7 Maintainability (NFR-MNT)

ID	Requirement	Pri	Ver	Rel	Status
NFR-MNT-01	Public functions and classes shall have type hints and docstrings.	S	I	R4	Planned
NFR-MNT-02	Code shall pass ruff lint and format checks with zero warnings in CI.	M	T	R1	In progress
NFR-MNT-03	Module dependencies shall flow in one direction (parser, flows, features, detectors, alerts, api) with no import cycles.	S	I	R2	Planned
NFR-MNT-04	Documentation shall include architecture, parser limits, configuration reference and a contributor guide.	M	I	R4	Planned

8.8 Portability and interoperability (NFR-PRT)

ID	Requirement	Pri	Ver	Rel	Status
NFR-PRT-01	The system shall support Python 3.10, 3.11 and 3.12, verified by CI.	M	T	R1	In progress
NFR-PRT-02	Offline analysis shall run on Windows 10/11, Ubuntu 22.04 or later (including Kali) and macOS 13 or later; live capture shall run on Windows (Npcap) and Linux (libpcap).	M	T	R4	Planned
NFR-PRT-03	When the optional C++ extension is missing or fails to import, the pure-Python path shall be used automatically with identical results.	S	T	R4	Planned
NFR-PRT-04	Captures written by Wireshark and tcpdump shall be readable, and captures written by the project shall open in Wireshark.	M	T	R1	Implemented
NFR-PRT-05	Exported alerts shall be UTF-8 JSON Lines, one object per line, ingestible by common log tools.	S	T	R2	Planned

8.9 Scalability and resource bounds (NFR-SCL)

ID	Requirement	Pri	Ver	Rel	Status
NFR-SCL-01	Memory shall be bounded: flow table limit, per-host window expiry and alert queue limit; no structure shall grow without bound with input size.	M	T	R2	Planned
NFR-SCL-02	The system targets a single node; documented limits shall state maximum sustained packets per second and active flows.	S	I	R4	Planned

8.10 Quality attribute priorities

When qualities conflict, the order of importance is: (1) correctness and security of parsing and detection, (2) explainability, (3) low false-alarm rate, (4) reliability under bad input, (5) performance, (6) portability, (7) usability polish. Example: a slower, bounded parser is preferred over a fast parser that can be crashed by a malformed packet.

9. Verification and acceptance

9.1 Test strategy

Level	What is tested	Tools and artefacts
Unit	Each function: header decoding, flag handling, flow timeouts, feature formulas, detector thresholds	pytest, synthetic frames built with struct (tests/helpers.py)
Cross-check	Parser output against an independent implementation on real traffic	scapy comparison on tests/fixtures/sample_web.pcap
Integration	Pipeline from pcap to stored alerts; configuration precedence; CLI exit codes	pytest with temporary directories and SQLite files
System	Attacks in the Docker lab versus labelled results	lab scripts, evaluation tool
Performance	Packets per second per stage, peak memory, parse speed-up with C++	benchmark tool, reference machine
Robustness	Fuzzing with random and mutated frames; truncated and oversized records	pytest parametrisation, hypothesis (optional)
Security	Safe YAML, model checksum, API binding, privacy of fixtures	unit tests, dependency alerts, manual review
Static	Lint, format, type hints, hooks	ruff, pre-commit, CI

9.2 Test data

Source	Use	Notes
Synthetic frames	Unit tests	Exact control of every header byte; no privacy risk
Anonymised fixture (under 1 MB)	Regression and cross-check	Generated by scripts/scrub_pcap.py; verified by tests/test_fixture.py
Docker lab captures	Detection accuracy	Labelled by attacker address and time range
CIC-IDS2017 / UNSW-NB15	ML training and evaluation	Downloaded locally; not committed; cited

9.3 Acceptance tests

ID	Test	Pass criterion	Rel
AT-01	Run flowsentinel analyze on the fixture	Exit code 0; summary shows packets, flows and zero alerts	R1
AT-02	Parser parity test (pytest cross-check)	All compared fields equal scapy for every IPv4 packet	R1
AT-03	Nmap SYN scan of 1,000 ports, 20 runs in the lab	Port-scan alert in at least 19 of 20 runs (recall 0.95)	R2
AT-04	hping3 SYN flood for 10 s, 20 runs	SYN-flood alert in at least 19 of 20 runs	R2
AT-05	Hydra SSH brute force, 20 runs	Brute-force alert in at least 19 of 20 runs	R2
AT-06	30 minutes of benign lab traffic	At most 0.1% of source-host windows raise an alert	R2
AT-07	Inspect every alert from AT-03 to AT-05	Each has reasons with measured value and threshold	R2
AT-08	Run train and evaluate	Report with confusion matrix, ROC-AUC, PR-AUC, generalisation gap; macro F1 0.85 on time-aware split	R3
AT-09	Replay lab traffic live	Rule alerts within 2 s; drop counter shown	R4
AT-10	Fuzz 10,000 mutated frames	No unhandled exception; parse errors counted	R4
AT-11	Benchmark on reference machine	At least 20,000 packets/s in Python; C++ parse stage 5 times faster; memory 512 MB at 100,000 flows	R4
AT-12	API and dashboard check	OpenAPI valid; new alert visible within 5 s; token enforced for non-local bind	R4
AT-13	Clean virtual environment install and README quick start	Five commands or fewer to a first analysis	R4

9.4 Definition of done (every release)

- All Must requirements for the release are Implemented and verified; Should items are done or deferred with a reason in the changelog.
- CI is green on Python 3.10, 3.11 and 3.12; pre-commit passes; no lint or format warnings.
- Documentation updated (README, docs/, configuration reference); release tagged with Semantic Versioning and release notes.
- No raw or personal captures in the repository; privacy test passes.

10. Traceability

10.1 Objectives to requirement groups

Objective	Primary requirement groups	Primary acceptance tests
O1 Detect attacks	FR-FEA, FR-DET, FR-ML, NFR-ACC	AT-03 to AT-06, AT-08
O2 Explain alerts	FR-EXP, FR-ALR, FR-UI	AT-07
O3 Reproducibility	FR-LAB, FR-EVL, FR-ML-03, FR-FEA-05	AT-03 to AT-08
O4 Speed	FR-PAR-11, NFR-PERF, NFR-SCL	AT-11
O5 Engineering quality	FR-DEV, NFR-MNT, NFR-PRT	AT-02, AT-13, CI results

10.2 Requirement groups to implementation and verification artefacts

Group	Implementation (target path)	Verification artefact	Rel
FR-CAP (file reader)	parser/pcap.py	tests/test_pcap.py, test_fixture_crosscheck.py	R1
FR-CAP (live)	capture/live.py	integration test, AT-09	R4
FR-PAR	parser/packet.py, parser/frames.py	tests/test_packet.py, test_frames.py, cross-check	R1
FR-FLW	flows/table.py	tests/test_flows.py	R1, R2
FR-FEA	features/*.py	tests/test_features.py	R1 to R3
FR-DET	detectors/rules/*.py	tests/test_detectors.py, AT-03 to AT-06	R2
FR-ML	detectors/ml/*.py	tests/test_ml.py, AT-08	R3
FR-EXP	explain/*.py	tests/test_explain.py, AT-07	R2, R3
FR-ALR	alerts/*.py	tests/test_alerts.py	R2
FR-CLI	cli.py	tests/test_cli.py, AT-01	R1 to R4
FR-API, FR-UI	api/, dashboard/	tests/test_api.py, AT-12	R4
FR-CFG	config.py	tests/test_config.py	R2
FR-LAB	lab/	AT-03 to AT-06 (lab runs)	R2
FR-EVL	scripts/evaluate.py, benchmark.py	AT-08, AT-11	R3, R4
FR-MON	monitoring.py, logging.py	tests/test_monitoring.py	R1 to R4
FR-DEV	.github/, .pre-commit-config.yaml, scripts/scrub_pcap.py	CI runs, tests/test_fixture.py	R1
FR-PKG	pyproject.toml	AT-13	R4
NFR groups	cross-cutting	benchmarks, fuzz tests, reviews (see Ver column)	R1 to R4

Appendix A. Worked numerical examples

These examples show where the thresholds in the requirements come from. Every number can be reproduced by hand or with the tests.

A.1 Decoding a TCP SYN by hand

A SYN from 10.0.0.1:51000 to 93.184.216.34:80 with sequence number 1000.

IPv4 bytes	Meaning
`45`	Version 4; header length 5 words x 4 = 20 bytes
`0028`	Total length = 0x28 = 40 bytes
`4000`	Don't-Fragment flag set; fragment offset 0
`40 06`	TTL 64; protocol 6 = TCP
`0a000001`	10.0.0.1
`5db8d822`	93, 184, 216, 34 = 93.184.216.34

TCP bytes	Meaning
`c738 0050`	Source port 0xc738 = 49152 + 1792 + 56 = 51000; destination port 80
`000003e8`	Sequence number 1000
`5002`	Data offset 5 words = 20 bytes; flags 0x02 = SYN
`faf0`	Window 0xfaf0 = 64240

Payload length = IP total length - IP header - TCP header = 40 - 20 - 20 = **0**. The frame is 14 + 40 = 54 bytes, but Ethernet pads it to 60, so 6 zero bytes follow. Counting frame bytes would invent a 6-byte payload; FR-PAR-05 forbids it. A SYN+ACK carries flags 0x12 = 0001 0010 = ACK (0x10) + SYN (0x02).

A.2 Port entropy separates normal hosts from scanners

Source host in a 10 s window	Distinct ports	Entropy H (bits)	Port-scan rule (50 ports and 4.0 bits)
Normal: 90 connections to port 443, 10 to port 53	2	0.47	No (2 ports, 0.47 bits)
Scanner: 100 connections, each to a different port	100	6.64	Yes (100 >= 50 and 6.64 >= 4.0)
Smaller scan: 50 connections, each to a different port	50	5.64	Yes (50 >= 50 and 5.64 >= 4.0)
Busy server client: 60 connections to 55 ports, skewed to port 443	55	about 2.3	No (entropy below 4.0 although ports >= 50)

Formula: H = -[sum] p log2 p. For the normal host: 0.9 x log2(1/0.9) + 0.1 x log2(1/0.1) = 0.9 x 0.152 + 0.1 x 3.322 = 0.137 + 0.332 = 0.47 bits. Requiring *both* a port count and an entropy value (FR-DET-01) removes false alarms from hosts that touch many ports but mostly the same one.

A.3 Why accuracy is not enough

Test set of 10,000 flows with 200 attacks. Operating point 1 uses a loose threshold; operating point 2 a tighter one.

	TP	FN	FP	TN	Accuracy	Precision	Recall	F1	FPR
Point 1	180	20	300	9,500	0.968	0.375	0.900	0.529	3.06%
Point 2	160	40	40	9,760	0.992	0.800	0.800	0.800	0.41%

Point 1 looks excellent on accuracy (96.8%) yet 62.5% of its alerts are false (precision 0.375). Point 2 loses 10 points of recall but cuts false alarms from 300 to 40, which is why NFR-ACC-04 requires the confusion matrix with every result and FR-ML-08 lets the user tune the threshold.

A.4 Isolation Forest anomaly score

Score s(x) = 2^(-E[h(x)] / c(n)), with c(n) = 2 H(n-1) - 2(n-1)/n and H(i) = ln(i) + 0.5772. For n = 256 samples per tree: H(255) = 6.1185, so c(256) = 2 x 6.1185 - 2 x 255/256 = **10.245**.

Average path length E[h(x)]	Exponent -E[h]/c(n)	Score s(x)	Reading
3 splits (isolated quickly)	-0.293	0.82	Likely anomaly (close to 1)
8 splits	-0.781	0.58	Borderline
c(n) = 10.2 splits	-1.000	0.50	Normal

A.5 Capacity and memory planning

Packets per second = bit rate / (average packet size x 8).

Case	Calculation	Result
Full 1 Gbit/s link, 500-byte packets	1,000,000,000 / (500 x 8)	250,000 packets/s
Python target (NFR-PERF-01)	20,000 x 500 x 8	80 Mbit/s = 8% of a 1 Gbit/s link
C++ parser target	100,000 x 500 x 8	400 Mbit/s = 40% of 1 Gbit/s
Flow table memory (assumed 1 KB per flow)	100,000 x 1 KB	about 100 MB, 5 times below the 512 MB limit (NFR-PERF-04)
Reference machine	4 cores, 8 GB RAM, SSD, Python 3.12	Used for all benchmark figures

The memory figure is an estimate to be replaced by measurement at R4. The conclusion for the architecture is that version 1.0 suits monitored segments up to a few hundred Mbit/s, not core links, which is stated as a limit (NFR-SCL-02).

A.6 False-alarm volume and analyst workload

False positive rate	False alerts per 1,000,000 benign flows	Triage time at 2 min each
1.00%	10,000	333.3 hours
0.10%	1,000	33.3 hours
0.01%	100	3.3 hours

At 1% a single day of traffic produces more triage work than a team can do. Hence the targets of 0.1% benign host-windows (NFR-ACC-02), the cooldown suppression in FR-DET-08, and threshold tuning to a target FPR in FR-ML-08.

A.7 Drift detection with the Population Stability Index

PSI = sum over bins of (actual - expected) x ln(actual / expected). Rule of thumb: below 0.1 stable, 0.1 to 0.25 moderate, above 0.25 significant. FR-MON-02 warns above 0.25.

Case	Expected bins	Actual bins	Calculation	PSI	Action
Similar traffic	0.5 / 0.3 / 0.2	0.4 / 0.3 / 0.3	(-0.1)(ln 0.8) + 0 + (0.1)(ln 1.5) = 0.0223 + 0.0405	0.063	Stable, no warning
Shifted traffic	0.5 / 0.3 / 0.2	0.2 / 0.3 / 0.5	(-0.3)(ln 0.4) + 0 + (0.3)(ln 2.5) = 0.2749 + 0.2749	0.550	Warn: retrain or review

A.8 Fixture file size

A classic pcap file is a 24-byte header plus, per packet, a 16-byte record header and the captured bytes. For 150 packets averaging 420 bytes: 150 x 420 = 63,000; 150 x 16 = 2,400; header 24; total **65,424 bytes (about 64 KB)**. The 1 MB limit (FR-DEV-05) would allow about 1,048,576 / 436 = 2,400 such packets, so a few seconds of controlled traffic is more than enough.

A.9 Flow key and timeouts

Canonical key. Packets 10.0.0.1:51000 -> 93.184.216.34:80 and 93.184.216.34:80 -> 10.0.0.1:51000 both map to key (6, (10.0.0.1, 51000), (93.184.216.34, 80)) because endpoints are ordered by numeric address, then port. The initiator is the source of the first packet, so the first direction is 'forward'.

Idle timeout 60 s. Packets of one 5-tuple arrive at t = 0, 1, 2 and 70 s. The gap 70 - 2 = 68 s exceeds 60 s, so flow 1 (3 packets, duration 2 s, reason idle) ends and the packet at 70 s starts flow 2. **Active timeout 120 s.** A download starting at t = 0 is emitted at t = 120 s (reason active) and continues as a new flow, so very long conversations still produce timely features.

Appendix B. Sample configuration (defaults)

```
capture:
  interface: null           # set for live mode
  bpf_filter: "ip"
flows:
  idle_timeout_s: 60
  active_timeout_s: 120
  max_active_flows: 100000
features:
  host_window_s: 10
detectors:
  cooldown_s: 60
  port_scan:    {enabled: true, min_unique_ports: 50, window_s: 10, min_entropy_bits: 4.0}
  syn_flood:    {enabled: true, syn_rate_per_s: 200, max_completion_ratio: 0.10}
  brute_force:  {enabled: true, ports: [21, 22, 23, 3389], min_connections: 20, window_s: 60}
  host_sweep:   {enabled: true, min_unique_hosts: 20, window_s: 10}
ml:
  enabled: false
  model_path: models/rf_v1.joblib
  threshold: 0.80
  explain_top_k: 5
alerts:
  database: flowsentinel.db
  retention_days: 30
api:
  host: 127.0.0.1
  port: 8000
  token: null
logging:
  level: INFO
  json: false
```

Appendix C. Sample alert and API payloads

Rule alert

```
{
  "id": 1042,
  "timestamp": "2027-01-05T10:15:32.418Z",
  "type": "port_scan",
  "severity": "high",
  "detector": "rule",
  "score": null,
  "src_ip": "10.0.0.5",
  "dst_ip": "10.0.0.20",
  "protocol": "tcp",
  "flow_id": null,
  "reasons": [
    "unique destination ports = 100 (threshold >= 50) in 10 s",
    "destination port entropy = 6.64 bits (threshold >= 4.0)",
    "SYN-only ratio = 0.97"
  ],
  "suppressed_count": 12,
  "model_version": null
}
```

ML alert (reasons from SHAP)

```
{
  "id": 1043, "type": "ml_attack", "severity": "medium", "detector": "ml",
  "score": 0.91, "src_ip": "10.0.0.5", "dst_ip": "10.0.0.20", "dst_port": 22,
  "flow_id": "7c1a9e02", "model_version": "rf_v1",
  "reasons": [
    "many short connections: packets per second is very high (+0.31)",
    "SYN count is much larger than normal (+0.22)",
    "very small average packet size (+0.12)",
    "few payload-carrying packets (+0.09)",
    "long gap before the last packet is not typical here (-0.04)"
  ]
}
```

GET /alerts?severity=high&limit=1

```
{"items": [{"id": 1042, "type": "port_scan", "severity": "high", "src_ip": "10.0.0.5"}],
 "total": 7, "limit": 1, "offset": 0}
```

Appendix D. Open issues and decisions to confirm

ID	Issue	Planned resolution	Owner / when
OI-01	IPv6 is not parsed; IPv6 addresses are not anonymised by the scrub tool	Add IPv6 parsing and scrubbing after 1.0; until then captures must be filtered with 'ip' (FR-PAR-10)	Later
OI-02	pcapng input is rejected	Convert or save as pcap; consider reading pcapng (FR-CAP-08)	Later
OI-03	pcap reader does not yet bound the record length	Add 262,144-byte cap with ParseError (FR-CAP-09)	R4 hardening
OI-04	scapy is currently a hard dependency; its licence is GPL-2.0	Move to optional capture extra, keep core scapy-free (NFR-ETH-05, FR-PKG-02)	R4
OI-05	Thresholds and accuracy targets are initial estimates	Tune on lab data at R2; revise ACC targets at R3 after the first evaluation	R2, R3
OI-06	Dashboard technology (Streamlit or a small static page with JavaScript)	Decide at the start of R4; requirement FR-UI-05 (no CDN) constrains the choice	R4
OI-07	Timestamps are floats (about 1 microsecond precision)	Consider integer nanoseconds if sub-microsecond ordering matters	R3
OI-08	No TCP stream reassembly or IP fragment reassembly	Out of scope for 1.0; document limits	Later
OI-09	Guidance for SPAN-port deployment and sensor sizing	Add to documentation with the benchmark results	R4

Appendix E. Requirements summary

Group	Total	Must	Should	Could	Implemented	In progress	Planned
FR-CAP	9	6	2	1	4	0	5
FR-PAR	11	9	2	0	9	0	2
FR-FLW	9	7	2	0	0	0	9
FR-FEA	7	6	1	0	0	0	7
FR-DET	8	6	2	0	0	0	8
FR-ML	10	10	0	0	0	0	10
FR-EXP	6	4	2	0	0	0	6
FR-ALR	7	5	1	1	0	0	7
FR-CLI	7	5	2	0	0	0	7
FR-API	7	4	2	1	0	0	7
FR-UI	5	2	3	0	0	0	5
FR-CFG	4	4	0	0	0	0	4
FR-LAB	5	4	1	0	0	0	5
FR-EVL	5	3	2	0	0	0	5
FR-MON	3	2	1	0	0	0	3
FR-DEV	10	6	4	0	2	4	4
FR-PKG	3	1	1	1	0	0	3
NFR-PERF	7	3	3	1	0	0	7
NFR-REL	5	4	1	0	0	0	5
NFR-ACC	5	4	1	0	1	0	4
NFR-SEC	8	8	0	0	0	1	7
NFR-ETH	5	4	1	0	0	0	5
NFR-USE	4	1	2	1	0	0	4
NFR-MNT	4	2	2	0	0	1	3
NFR-PRT	5	3	2	0	1	1	3
NFR-SCL	2	1	1	0	0	0	2
All	**161**	**114**	**41**	**6**	**17**	**7**	**137**

Reading the numbers. 161 requirements in total; 114 are Must. Status is measured on 27 October 2026: the parser and pcap reader are implemented and tested, the engineering set-up (CI, pre-commit, anonymised fixture) is being merged, and everything else is planned for releases R1 to R4.

End of document.